

Pyetje & Përgjigje të Përgjithshme

Lab Course 1 (Programim) | Viti Akademik 2025/2026 | 50 pyetje provimi

1. Koncepte të Përgjithshme të Zhvillimit

Pyetja 1: Çfarë është HTTP dhe cilat janë metodat kryesore?

Përgjigja:

HTTP (HyperText Transfer Protocol) është protokoli i komunikimit në Web midis klientit (shfletuesit) dhe serverit. Funksionon mbi modelin kërkesë-përgjigje (request/response) dhe është pa gjendje (stateless): çdo kërkesë trajtohet në mënyrë të pavarur.

Metodat kryesore:

- GET: lexon (merr) të dhëna nga serveri, nuk duhet të ndryshojë gjendjen
- POST: krijon një burim të ri në server
- PUT: zëvendëson plotësisht një burim ekzistues
- PATCH: përditëson pjesërisht një burim ekzistues
- DELETE: fshin një burim në server

Pyetja 2: Çfarë janë HTTP status codes dhe cilat janë më të zakonshmet?

Përgjigja:

HTTP status codes janë kode numerike treshifrore që serveri kthen për të treguar rezultatin e një kërkesë. Ndahen në pesë familje sipas shifrës së parë (1xx informative, 2xx sukses, 3xx ridrejtime, 4xx gabime të klientit, 5xx gabime të serverit).

Më të zakonshmet:

- 200 OK: kërkesa u krye me sukses
- 201 Created: u krijua një burim i ri (zakonisht pas POST)
- 400 Bad Request: kërkesa ka të dhëna të pavlefshme
- 401 Unauthorized: mungon ose është i pavlefshëm autentifikimi
- 403 Forbidden: përdoruesi është i autentifikuar por nuk ka të drejta
- 404 Not Found: burimi i kërkuar nuk ekziston
- 500 Internal Server Error: gabim i papritur në server

Pyetja 3: Çfarë është një REST API dhe cilat janë parimet e tij?

Përgjigja:

REST (Representational State Transfer) është një stil arkitektonik për ndërtimin e API-ve mbi HTTP, ku të dhënat ekspozohen si burime (resources) të identifikueshme me URL.

Parimet kryesore:

- Client-Server: ndarje e qartë e përgjegjësiwe midis klientit dhe serverit

- Stateless: çdo kërkesë përmban të gjithë informacionin e nevojshëm
- Cacheable: përgjigjet mund të ruhen në cache
- Ndërfaqe uniforme: përdor metodat standarde HTTP (GET, POST, PUT, DELETE)
- Burime me URL të qarta, p.sh. /api/users/1
- Përfaqësim i të dhënave zakonisht në JSON

Shembull: GET /api/users/1 kthen përdoruesin me id=1.

Pyetja 4: Çfarë është JSON dhe pse përdoret gjerësisht në API?

Përgjigja:

JSON (JavaScript Object Notation) është një format teksti i lehtë për shkëmbim të dhënash midis sistemeve. Bazohet në çifte çelës-vlerë (key-value) dhe është i lehtë për t'u lexuar nga njerëzit dhe nga makinat.

Pse përdoret në API:

- Sintaksë e thjeshtë dhe e qartë
- I pavarur nga gjuha e programimit (mbështetet nga të gjitha gjuhët moderne)
- Më kompakt se XML
- Përputhet natyrshëm me objektet në JavaScript

Shembull:

```
{
  "id": 1,
  "name": "Blerim",
  "active": true
}
```

Pyetja 5: Çfarë është OOP dhe cilat janë 4 shtyllat e tij?

Përgjigja:

OOP (Object-Oriented Programming) është një paradigmë programimi që organizon kodin rreth objekteve, të cilat janë instanca të klasave dhe përmbajnë të dhëna (atribute) dhe sjellje (metoda).

Katër shtyllat:

- Encapsulation (kapsulimi): fshehja e gjendjes së brendshme dhe ekspozimi vetëm i ndërfaqes publike (p.sh. fusha private me getter/setter)
- Inheritance (trashëgimia): një klasë merr veti dhe metoda nga një klasë prind, duke evituar përsëritjen e kodit
- Polymorphism (polimorfizmi): i njëjti emër metode sillet ndryshe sipas tipit konkret (override, overload)
- Abstraction (abstragimi): paraqitja vetëm e koncepteve thelbësore, duke fshehur detajet e implementimit (klasa abstrakte, interface)

Pyetja 6: Çfarë është MVC pattern?

Përgjigja:

MVC (Model-View-Controller) është një model arkitektonik që ndan një aplikacion në tri komponente me përgjegjësi të dallueshme:

- Model: përfaqëson të dhënat dhe logjikën e biznesit; komunikon me bazën e të dhënave
- View: shtresa e prezantimit (UI) që i shfaq të dhënat përdoruesit
- Controller: pranon kërkesat e përdoruesit, thërret Model-in dhe zgjedh View-n e duhur për përgjigje

Përfitimet: ndarje e qartë e përgjegjësi, kod më i mirëmbajtshëm dhe i testueshëm. Përdoret në framework si ASP.NET MVC, Spring MVC dhe Laravel.

Pyetja 7: Çfarë është Git dhe cilat janë operacionet bazë?

Përgjigja:

Git është një sistem i shpërndarë i kontrollit të versioneve që ruan historikun e ndryshimeve të kodit dhe lejon disa zhvillues të bashkëpunojnë mbi të njëjtin projekt.

Operacionet bazë:

- git clone: kopjon një repository nga remote në kompjuterin lokal
- git add: shton skedarët në staging area
- git commit: ruan ndryshimet me një mesazh përshkrues
- git push: dërgon commit-et lokale në remote (p.sh. GitHub)
- git pull: merr ndryshimet nga remote dhe i bashkon në degën lokale
- git branch: krijon ose liston degë
- git merge: bashkon ndryshimet e një dege në një tjetër

Pyetja 8: Çfarë është një branch dhe çfarë është merge konflikti?

Përgjigja:

Një branch (degë) është një linjë e pavarur zhvillimi që rrjedh nga një pikë e historisë së projektit. Lejon punë paralele në veçori të reja pa prekur degën kryesore (main ose master).

Merge konflikti ndodh kur Git nuk mund të bashkojë automatikisht ndryshimet, sepse dy degë kanë modifikuar të njëjtën pjesë të të njëjtit skedar. Zgjidhja kërkon ndërhyrje manuale: zhvilluesi hap skedarin, zgjedh cilat ndryshime të mbajë, fshin shenjat speciale (<<<<<<, =====, >>>>>>) dhe pastaj bën git add dhe git commit për të përfunduar bashkimin.

Pyetja 9: Çfarë është pull request (PR) dhe code review?

Përgjigja:

Pull Request (PR) është një kërkesë formale për të bashkuar ndryshimet e një dege në një degë tjetër (zakonisht në main). Krijohet në platforma si GitHub, GitLab ose Bitbucket dhe shërben si pikë diskutimi rreth ndryshimeve të propozuara.

Code review është procesi ku një ose disa zhvillues të tjerë lexojnë kodin e propozuar para se të bashkohet.

Synimet:

- Gjetja e gabimeve dhe problemeve të mundshme
- Siguria dhe performanca e kodit
- Pajtueshmëria me standardet e ekipit
- Ndarja e njohurive midis anëtarëve

PR zakonisht përfshin: titull, përshkrim, lidhje me task-un dhe checklist testimi.

Pyetja 10: Çfarë është Trello dhe si ndihmon në menaxhimin e detyrave?

Përgjigja:

Trello është një mjet vizual për menaxhim projektesh i bazuar në metodologjinë Kanban. Lejon ekipet të organizojnë punën në mënyrë të thjeshtë dhe transparente.

Elementet kryesore:

- Boards: tabela për një projekt ose ekip
- Lists: kolona që paraqesin faza (p.sh. To Do, In Progress, Done)
- Cards: kartëla që përfaqësojnë detyra individuale, lëvizin midis listave
- Labels: etiketa me ngjyra për kategorizim (bug, feature, urgent)
- Members: anëtarët e ekipit që janë caktuar për një kartëlë
- Due dates dhe checklists: afate dhe nën-detyra brenda kartëlës

Ndihmon në shikimin e gjendjes së projektit në një vështrim të vetëm.

Pyetja 11: Çfarë është një IDE dhe cili është ndryshimi nga një editor?

Përgjigja:

IDE (Integrated Development Environment) është një mjedis i integruar zhvillimi që përmban editor kodi, kompajler ose interpretues, debugger, menaxhues paketash dhe integrim me kontroll versionesh, të gjitha brenda një aplikacioni të vetëm.

Një editor është një program më i thjeshtë për redaktim teksti/kodi, që mund të zgjerohet me plugin për funksione shtesë.

Shembuj:

- IDE: IntelliJ IDEA (Java), Visual Studio (.NET), PyCharm (Python), Android Studio
- Editor: Sublime Text, Notepad++, Vim
- Mes të dyjave: VS Code, që është editor i lehtë por me ekosistem të pasur extension-esh që e afrojnë me një IDE.

Pyetja 12: Çfarë janë DevTools në shfletues dhe si përdoren për debug?

Përgjigja:

DevTools (Developer Tools) janë mjete të integruara në shfletuesit modernë (Chrome, Firefox, Edge) që ndihmojnë në inspektim, debug dhe optimizim të aplikacioneve web. Hapen zakonisht me F12 ose Ctrl+Shift+I.

Panelet kryesore:

- Elements: inspektim dhe ndryshim i HTML/CSS në kohë reale
- Console: shfaq mesazhe, gabime dhe lejon ekzekutim të JavaScript
- Network: shfaq të gjitha kërkesat HTTP, statusin, kohën dhe payload-in
- Sources: vendosje breakpoint-esh dhe ecje hap pas hapi nëpër JavaScript
- Application: shikim i localStorage, cookies, sessionStorage
- Performance: matje e shpejtësisë dhe identifikim i pengesave

Pyetja 13: Çfarë është Postman dhe pse përdoret?

Përgjigja:

Postman është një mjet desktop/web që përdoret për të testuar dhe dokumentuar API-të HTTP pa nevojë për të shkruar kod klienti.

Funksionet kryesore:

- Dërgim i kërkesave HTTP me metoda të ndryshme (GET, POST, PUT, DELETE)
- Vendosje e headers, parametrave dhe body në JSON
- Menaxhim i autentifikimit (Bearer Token, Basic Auth, OAuth)
- Organizim i kërkesave në Collections dhe Environments

- Krijim i testeve automatike me JavaScript
- Ndarje e koleksioneve me ekipin

Përdoret nga zhvilluesit backend për të verifikuar endpoint-et dhe nga zhvilluesit frontend për të kuptuar API-në para se ta integrojnë.

Pyetja 14: Çfarë është SDLC (Software Development Life Cycle)?

Përgjigja:

SDLC është procesi i strukturuar përmes të cilit kalon zhvillimi i një softueri, nga ideja deri te mirëmbajtja. Synon të prodhojë softuer cilësor në kohë dhe brenda buxhetit.

Fazat kryesore:

1. Mbledhja e kërkesave (Requirements): kuptohet çfarë do klienti
2. Analiza dhe planifikimi: vlerësohen burimet, koha, rreziqet
3. Dizajni: arkitektura e sistemit dhe baza e të dhënave
4. Implementimi (Coding): shkrimi i kodit
5. Testimi: verifikimi se softueri funksionon si pritet
6. Deployment: vënia në prodhim
7. Mirëmbajtja: rregullim bug-esh dhe shtim veçorish

Pyetja 15: Çfarë është Agile dhe cili është ndryshimi nga Waterfall?

Përgjigja:

Agile është një qasje iterative dhe inkrementale ndaj zhvillimit të softuerit, ku puna ndahet në cikle të shkurtra (zakonisht 2 javë) dhe produkti zhvillohet gradualisht me reagime të vazhdueshme nga klienti.

Waterfall është model linear ku fazat (kërkesa, dizajn, implementim, testim, deployment) kalojnë në mënyrë sekuenciale: faza tjetër fillon vetëm kur e mëparshmeja ka përfunduar.

Dallimet kryesore:

- Agile: fleksibël, mirëpret ndryshime, dorëzim i shpeshtë
- Waterfall: i ngurtë, ndryshimet pas planifikimit janë të vështira
- Agile: bashkëpunim i ngushtë me klientin gjatë gjithë projektit
- Waterfall: dokumentim i detajuar para se të fillojë kodimi

Pyetja 16: Çfarë është një Scrum sprint?

Përgjigja:

Scrum sprint është një periudhë e shkurtër dhe e fiksuar (zakonisht 1 deri 4 javë) gjatë së cilës ekipi punon për të dorëzuar një inkrement të përdorshëm të produktit. Është njësia themelore e punës në framework-un Scrum.

Ngjarjet kryesore brenda sprintit:

- Sprint Planning: ekipi zgjedh çfarë do të bëjë në sprint
- Daily Standup: takim i shkurtër ditor (15 min) për sinkronizim
- Sprint Review: prezantimi i punës së përfunduar para stakeholders
- Sprint Retrospective: reflektim mbi ç'shko mirë dhe çfarë të përmirësohet

Rolet kryesore: Product Owner, Scrum Master dhe Development Team.

Pyetja 17: Cilat janë principet SOLID?

Përgjigja:

SOLID është një grup prej pesë principesh dizajni në OOP që ndihmojnë në shkrimin e kodit të mirëmbajtshëm dhe të zgjeruar.

- S - Single Responsibility Principle: çdo klasë duhet të ketë vetëm një arsye për të ndryshuar (vetëm një përgjegjësi)
- O - Open/Closed Principle: klasat duhet të jenë të hapura për zgjerim, por të mbyllura për modifikim
- L - Liskov Substitution Principle: objektet e një klase fëmijë duhet të mund të zëvendësojnë objektet e klasës prind pa prishur sjelljen
- I - Interface Segregation Principle: më mirë shumë interface specifike sesa një i madh i përgjithshëm
- D - Dependency Inversion Principle: varësitë duhet të jenë mbi abstraksione, jo mbi implementime konkrete

Pyetja 18: Çfarë janë principet DRY dhe KISS?

Përgjigja:

DRY (Don't Repeat Yourself): çdo pjesë e logjikës duhet të ekzistojë në një vend të vetëm në kod. Përsëritja e kodit shton mundësinë e gabimeve dhe e bën mirëmbajtjen më të vështirë. Zgjidhja: nxjerrja e logjikës së përsëritur në funksione, klasa ose module të ripërdorshme.

KISS (Keep It Simple, Stupid): zgjidhjet duhet të mbahen sa më të thjeshta të jetë e mundur. Kodi i thjeshtë lexohet, testohet dhe mirëmbahet më lehtë. Evito abstragime të panevojshme, modele dizajni të tepruara dhe optimizime para kohe.

Të dy principet ndihmojnë në shkrimin e kodit më të pastër dhe më të qëndrueshëm në kohë.

Pyetja 19: Çfarë janë unit tests dhe pse janë të rëndësishëm?

Përgjigja:

Unit tests janë teste të automatizuara që verifikojnë sjelljen e njësive më të vogla të kodit (zakonisht një funksion ose metodë) në izolim nga pjesa tjetër e sistemit.

Pse janë të rëndësishëm:

- Zbulojnë gabimet herët në procesin e zhvillimit
- Shërbejnë si dokumentim i gjallë i sjelljes së pritur
- Mundësojnë refactoring me siguri (nëse testet kalojnë, sjellja ruhet)
- Reduktojnë kohën e debug-imit dhe regresioneve

```
Shembull (JavaScript me Jest):  
test('sum adds two numbers', () => {  
  expect(sum(2, 3)).toBe(5);  
});
```

Framework-e të zakonshme: JUnit (Java), NUnit/xUnit (.NET), Jest (JS), PyTest (Python).

Pyetja 20: Çfarë është dokumentacioni i kodit dhe pse është i rëndësishëm?

Përgjigja:

Dokumentacioni i kodit është informacioni shpjegues që shoqëron softuerin për të ndihmuar zhvilluesit dhe përdoruesit ta kuptojnë, ta përdorin dhe ta mirëmbajnë atë.

Llojet kryesore:

- Komente në kod: shpjegojnë 'pse' bëhet diçka, jo 'çfarë' (kjo duket nga vetë kodi)
- README: skedari hyrës i projektit me udhëzime për instalim, ekzekutim dhe kontribut

- API docs: dokumentim i endpoint-eve, parametrave dhe përgjigjeve (p.sh. Swagger/OpenAPI)
- Wiki ose docs site: dokumentim i thelluar mbi arkitekturën dhe vendimet teknike

Pse është i rëndësishëm:

- Lehtëson onboarding-un e zhvilluesve të rinj
- Redukton varësinë nga një person i vetëm që njeh sistemin
- Ndhmon në mirëmbajtje afatgjatë dhe shmang ripyetje të njëjta

2. Databaza

Pyetja 1: Çfarë është një Sistem Menaxhimi i Bazave të të Dhënave Relacionale (RDBMS) dhe cilët janë disa shembuj?

Përgjigja:

Një RDBMS (Relational Database Management System) është një softuer që ruan, menaxhon dhe lejon manipulimin e të dhënave të organizuara në tabela të lidhura midis tyre me anë të relacioneve. Të dhënat ruhen në rreshta dhe kolona, ndërsa komunikimi me bazën bëhet kryesisht përmes gjuhës SQL (Structured Query Language).

Shembuj të njohur:

1. Microsoft SQL Server (MSSQL): RDBMS komercial nga Microsoft, përdor T-SQL.
2. MySQL: RDBMS me burim të hapur, shumë i përdorur në aplikacionet web.
3. Oracle Database: RDBMS i fuqishëm komercial, përdor PL/SQL, shpesh në sisteme enterprise.
4. PostgreSQL: RDBMS me burim të hapur me veçori të avancuara.

Pyetja 2: Çfarë janë Primary Key, Foreign Key dhe Unique Key?

Përgjigja:

Primary Key është një kolonë (ose kombinim kolonash) që identifikon në mënyrë unike çdo rresht në një tabelë. Nuk lejon vlera NULL dhe duhet të jetë unike.

Foreign Key është një kolonë që referon Primary Key-n e një table tjetër; siguron integritetin referencial midis tabelave.

Unique Key garanton që vlerat në një kolonë të jenë unike, por ndryshe nga Primary Key, lejon një vlerë NULL.

Shembull:

```
CREATE TABLE Studentet (  
  id INT PRIMARY KEY,  
  email VARCHAR(100) UNIQUE,  
  fakulteti_id INT,  
  FOREIGN KEY (fakulteti_id) REFERENCES Fakultetet(id)  
);
```

Pyetja 3: Cilat janë llojet e relacioneve midis tabelave (One-to-One, One-to-Many, Many-to-Many)?

Përgjigja:

One-to-One (1:1): Një rresht i tabelës A lidhet me vetëm një rresht të tabelës B. P.sh. një Përdorues ka një Profil.

One-to-Many (1:N): Një rresht i tabelës A lidhet me shumë rreshta të tabelës B. P.sh. një Fakultet ka shumë

Studentë.

Many-to-Many (N:M): Shumë rreshta të A lidhen me shumë rreshta të B; realizohet me një tabelë ndërmjetëse (junction table). P.sh. Studentet dhe Lendet.

Shembull për N:M:

```
CREATE TABLE StudentLende (  
    student_id INT,  
    lende_id INT,  
    PRIMARY KEY (student_id, lende_id),  
    FOREIGN KEY (student_id) REFERENCES Studentet(id),  
    FOREIGN KEY (lende_id) REFERENCES Lendet(id)  
);
```

Pyetja 4: Cilat janë operacionet CRUD në SQL dhe si shkruhen?

Përgjigja:

CRUD janë katër operacionet bazë: Create, Read, Update, Delete. Në SQL korrespondojnë me INSERT, SELECT, UPDATE dhe DELETE.

Shembuj:

-- Create (INSERT)

```
INSERT INTO Studentet (id, emri, email) VALUES (1, 'Blerim', 'b@ubt.edu');
```

-- Read (SELECT)

```
SELECT id, emri FROM Studentet WHERE fakulteti_id = 2;
```

-- Update (UPDATE)

```
UPDATE Studentet SET email = 'i_ri@ubt.edu' WHERE id = 1;
```

-- Delete (DELETE)

```
DELETE FROM Studentet WHERE id = 1;
```

Kujdes: UPDATE dhe DELETE pa WHERE prekin të gjithë rreshtat e tabelës.

Pyetja 5: Çfarë janë JOIN-et në SQL (INNER, LEFT, RIGHT, FULL)?

Përgjigja:

JOIN-i kombinon rreshta nga dy ose më shumë tabela bazuar në një kusht (zakonisht Primary Key dhe Foreign Key).

INNER JOIN: kthen vetëm rreshtat që kanë përputhje në të dyja tabelat.

LEFT JOIN: kthen të gjithë rreshtat nga tabela e majtë dhe ata që përputhen nga e djathta; pa përputhje, vlerat janë NULL.

RIGHT JOIN: e kundërta e LEFT JOIN.

FULL JOIN: kthen të gjithë rreshtat nga të dyja tabelat, me NULL kur s'ka përputhje.

Shembull:

```
SELECT s.emri, f.titulli  
FROM Studentet s  
INNER JOIN Fakultetet f ON s.fakulteti_id = f.id;
```

Pyetja 6: Çfarë është një INDEX dhe kur duhet të krijohet?

Përgjigja:

Një INDEX është një strukturë ndihmëse e të dhënave (zakonisht B-Tree) që përshpejton kërkimin e rreshtave në një tabelë, ngjashëm me indeksin e një libri. Përmirëson performancën e SELECT, por ngadalëson INSERT, UPDATE dhe DELETE sepse indeksi duhet të mirëmbahet.

Krijohet zakonisht në:

1. Kolonat e përdorura shpesh në WHERE, JOIN ose ORDER BY.
2. Kolonat me selektivitet të lartë (shumë vlera unike).

Shembull:

```
CREATE INDEX idx_studentet_email ON Studentet(email);
```

Mos krijo indekse të panevojshme në tabela me shumë shkrime, sepse ulin performancën.

Pyetja 7: Çfarë është normalizimi dhe cilat janë format 1NF, 2NF dhe 3NF?

Përgjigja:

Normalizimi është procesi i organizimit të të dhënave në tabela për të eliminuar redundancën dhe për të ruajtur integritetin.

1NF (First Normal Form): çdo qelizë ka një vlerë atomike (jo lista, jo grupe), dhe çdo rresht është unik.

2NF: është në 1NF dhe çdo kolonë jo-kyçe varet nga e gjithë Primary Key (eliminon varësitë e pjesshme).

3NF: është në 2NF dhe nuk ka varësi tranzitive, pra kolonat jo-kyçe nuk varen nga kolona të tjera jo-kyçe.

Shembull i shkeljes së 3NF: nëse tabela Studentet ka 'fakulteti_id' dhe 'fakulteti_emri', emri varet nga ID, jo nga studenti. Zgjidhja: ndaje 'fakulteti_emri' në tabelën Fakultetet.

Pyetja 8: Çfarë janë constraints në SQL (NOT NULL, CHECK, DEFAULT, UNIQUE, FOREIGN KEY)?

Përgjigja:

Constraints janë rregulla që zbatohen mbi të dhënat në kolona për të siguruar saktësinë dhe integritetin.

NOT NULL: kolona nuk mund të ketë vlerë NULL.

CHECK: vlera duhet të plotësojë një kusht logjik.

DEFAULT: vlerë e parazgjedhur kur nuk jepet asnjë.

UNIQUE: vlerat në kolonë duhet të jenë unike.

FOREIGN KEY: referenca në Primary Key të një table tjetër.

Shembull:

```
CREATE TABLE Studentet (  
    id INT PRIMARY KEY,  
    emri VARCHAR(100) NOT NULL,  
    mosha INT CHECK (mosha >= 18),  
    statusi VARCHAR(20) DEFAULT 'aktiv',  
    email VARCHAR(100) UNIQUE  
);
```

Pyetja 9: Çfarë është një Stored Procedure dhe cili është ndryshimi nga një funksion?

Përgjigja:

Stored Procedure është një bllok kodi SQL i ruajtur në server që ekzekuton një grup operacionesh; mund të pranojë parametra IN dhe OUT, mund të modifikojë të dhëna dhe nuk është i detyruar të kthejë një vlerë.

Funksioni (User Defined Function) duhet të kthejë gjithmonë një vlerë, përdoret brenda shprehjeve SQL (p.sh. SELECT) dhe zakonisht nuk modifikon të dhënat.

Shembull (MSSQL):

```
CREATE PROCEDURE GetStudentet @fakulteti_id INT AS  
BEGIN  
    SELECT * FROM Studentet WHERE fakulteti_id = @fakulteti_id;  
END;
```

```
EXEC GetStudentet @fakulteti_id = 2;
```

Pyetja 10: Çfarë janë transaksionet dhe cilat janë vetitë ACID?

Përgjigja:

Një transaksion është një sekuencë operacionesh SQL që trajtohen si një njësi e vetme: ose ekzekutohen të gjitha, ose asnjë.

Vetitë ACID:

Atomicity: të gjitha operacionet brenda transaksionit kryhen ose asnjëri.

Consistency: baza kalon nga një gjendje e vlefshme në një tjetër të vlefshme.

Isolation: transaksionet e njëkohshme nuk ndikojnë në njëri-tjetrin.

Durability: pasi transaksioni kryhet (COMMIT), ndryshimet janë të përhershme edhe në rast crash-i.

Shembull:

```
BEGIN TRANSACTION;  
UPDATE Llogarite SET balanca = balanca - 100 WHERE id = 1;  
UPDATE Llogarite SET balanca = balanca + 100 WHERE id = 2;  
COMMIT; -- ose ROLLBACK në rast gabimi
```

Pyetja 11: Cilat janë ndryshimet kryesore midis MSSQL, MySQL dhe Oracle?

Përgjigja:

Të tre janë RDBMS, por kanë sintaksa dhe veçori specifike.

Auto-incrementi i kolonave:

MSSQL: IDENTITY(1,1)

id INT IDENTITY(1,1) PRIMARY KEY

MySQL: AUTO_INCREMENT

id INT AUTO_INCREMENT PRIMARY KEY

Oracle: SEQUENCE (ose IDENTITY në versionet e reja)

CREATE SEQUENCE seq_studentet START WITH 1 INCREMENT BY 1;

INSERT INTO Studentet VALUES (seq_studentet.NEXTVAL, 'Blerim');

Ndryshime të tjera:

1. Gjuhët procedurale: T-SQL (MSSQL), PL/SQL (Oracle), SQL standarde + procedure (MySQL).
2. LIMIT (MySQL) kundrejt TOP (MSSQL) kundrejt ROWNUM/FETCH FIRST (Oracle).
3. Tipet e të dhënave: NVARCHAR (MSSQL), VARCHAR2 (Oracle), VARCHAR (MySQL).

Pyetja 12: Çfarë janë SQL Injections dhe si parandalohen?

Përgjigja:

SQL Injection është një sulm ku një përdorues keqdashës fut kod SQL brenda fushave të input-it për të manipuluar query-n origjinal, duke arritur të lexojë, ndryshojë ose fshijë të dhëna pa autorizim.

Shembull i kodit të rrezikshëm (mos e bëj kështu):

```
query = "SELECT * FROM Users WHERE email = '" + email + "'";  
// Nëse email = ' OR '1'='1, kthehen të gjithë përdoruesit.
```

Mënyrat e parandalimit:

1. Parameterized queries (prepared statements): vlerat trajtohen si parametra, jo si pjesë e SQL-it.
cmd.Parameters.AddWithValue('@email', email);
2. Përdorimi i ORM-ve (Entity Framework, Hibernate, Sequelize) që përdorin parametra automatikisht.

4. Përdorimi i përdoruesve të bazës me të drejta minimale (least privilege).

3. Siguria dhe Autentifikimi

Pyetja 1: Çfarë është autentifikimi dhe cili është ndryshimi nga autorizimi?

Përgjigja:

Autentifikimi (authentication) është procesi i verifikimit se kush është përdoruesi, zakonisht përmes kredencialeve si username dhe password ose token. Autorizimi (authorization) është procesi që përcakton se çfarë lejohet të bëjë një përdorues i autentifikuar, p.sh. nëse ka të drejtë të hyjë në një burim ose të kryejë një veprim.

Shembull i thjeshtë:

Autentifikimi: 'Kush je ti?' => login me email dhe password.

Autorizimi: 'Çfarë mund të bësh?' => a ke rolin admin për të fshirë një përdorues.

Pyetja 2: Çfarë është JWT dhe cila është struktura e tij?

Përgjigja:

JWT (JSON Web Token) është një standard i hapur për transmetimin e të dhënave të nënshkruara në mënyrë të sigurt midis palëve. Përbëhet nga tri pjesë të ndara me pikë: header.payload.signature.

1. Header: lloji i tokenit dhe algoritmi (p.sh. HS256).
2. Payload: claims si userId, role, exp (koha e skadimit).
3. Signature: nënshkrimi që siguron integritetin, i krijuar me një secret key në server.

Shembull:

eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJF9.dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk

Pyetja 3: Si rrjedh procesi i login me JWT?

Përgjigja:

Rrjedha standarde është si më poshtë:

1. Klienti dërgon credentials (email + password) në endpoint-in /login.
2. Serveri verifikon kredencialet kundrejt bazës së të dhënave.
3. Nëse janë të sakta, serveri lëshon një JWT të nënshkruar me secret key.
4. Klienti e ruan tokenin dhe e dërgon në çdo kërkesë në header-in 'Authorization: Bearer <token>'.
5. Serveri verifikon nënshkrimin dhe skadimin para se të lejojë qasjen.

Shembull header:

Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJ1c2VhcnQiaW9jF9.dBiftJeZ4...

Pyetja 4: Çfarë janë access token dhe refresh token dhe kur përdoren?

Përgjigja:

Access token është një token me jetëgjatësi të shkurtër (p.sh. 15 minuta) që përdoret për të hyrë në burimet e mbrojtura. Refresh token ka jetëgjatësi më të gjatë (p.sh. 7 ditë) dhe përdoret vetëm për të marrë një access token të ri kur i vjetri skadon, pa e detyruar përdoruesin të bëjë login përsëri.

Avantazhi: nëse access token vidhet, dëmi është i kufizuar në kohë. Refresh token zakonisht ruhet në një httpOnly cookie dhe revokohet në server nëse ka aktivitet të dyshimtë.

Shembull endpoint: POST /auth/refresh => kthen access token të ri.

Pyetja 5: Si ruhen tokenat në klient dhe pse httpOnly cookie është më e sigurt se localStorage?

Përgjigja:

Dy mënyrat kryesore për ruajtjen e tokenit në klient janë:

1. localStorage: i thjeshtë për t'u përdorur, por është i aksesueshëm nga JavaScript, prandaj çdo skript i injektuar përmes XSS mund ta lexojë tokenin.
2. httpOnly cookie: cookie që nuk lexohet nga JavaScript. Browser-i e dërgon automatikisht në çdo kërkesë te i njëjti domain.

httpOnly cookie kombinuar me flag-et 'Secure' (vetëm HTTPS) dhe 'SameSite=Strict' ofron mbrojtje më të mirë ndaj XSS dhe CSRF.

Shembull (Express):

```
res.cookie('token', jwt, { httpOnly: true, secure: true, sameSite: 'strict' });
```

Pyetja 6: Çfarë është hash-imi i fjalëkalimit me bcrypt dhe pse nuk ruhen kurrë fjalëkalimet në plain-text?

Përgjigja:

Fjalëkalimet nuk ruhen kurrë në plain-text sepse, nëse baza e të dhënave kompromentohet, sulmuesi do të kishte qasje direkte në llogaritë e të gjithë përdoruesve. Bcrypt është algoritëm hash-imi i njëanshëm i krijuar posaçërisht për fjalëkalime.

Karakteristikat e bcrypt:

1. Salt: një varg i rastësishëm që i shtohet fjalëkalimit para hash-imit, që dy përdorues me të njëjtin password të kenë hash të ndryshëm.
2. Rounds (cost factor): sa herë përsëritet algoritmi (p.sh. 10-12), që e bën sulmin brute-force të ngadalshëm.

Shembull (Node.js):

```
const hash = await bcrypt.hash(password, 12);  
const ok = await bcrypt.compare(password, hash);
```

Pyetja 7: Çfarë është CORS dhe si konfigurohet në backend?

Përgjigja:

CORS (Cross-Origin Resource Sharing) është mekanizëm i browser-it që kontrollon nëse një faqe nga një origin (p.sh. http://localhost:3000) lejohet të bëjë kërkesa në një origin tjetër (p.sh. http://localhost:5000). Pa CORS, browser-i bllokoi kërkesat cross-origin për arsye sigurie.

Serveri duhet të kthejë header-at e duhur, p.sh. 'Access-Control-Allow-Origin', 'Access-Control-Allow-Methods' dhe 'Access-Control-Allow-Headers'.

Shembull (Express):

```
const cors = require('cors');  
app.use(cors({ origin: 'http://localhost:3000', credentials: true }));
```

Pyetja 8: Çfarë është SQL injection dhe si parandalohet?

Përgjigja:

SQL injection është sulm ku sulmuesi fut kod SQL të dëmshëm përmes input-it të përdoruesit për të manipuluar query-të e bazës së të dhënave (p.sh. të fshijë tabela ose të lexojë të dhëna të mbrojtura).

Shembull i cenueshëm:

```
query = "SELECT * FROM users WHERE email = '" + email + "'";
```

Nëse email = "' OR '1'='1", query-ja kthen të gjithë përdoruesit.

Parandalimi:

1. Përdor parameterized queries (prepared statements) që ndajnë kodin SQL nga të dhënat.
2. Përdor një ORM si Sequelize, TypeORM, Entity Framework ose Eloquent.
3. Validimi i input-it dhe parimi i privilegjit minimal për përdoruesin e DB-së.

Shembull i sigurt:

```
db.query('SELECT * FROM users WHERE email = ?', [email]);
```

Pyetja 9: Çfarë është XSS (Cross-Site Scripting) dhe si mbrohem prej tij?

Përgjigja:

XSS është sulm ku sulmuesi injekton kod JavaScript të dëmshëm në një faqe web, i cili pastaj ekzekutohet në browser-in e viktimës. Mund të përdoret për të vjedhur tokena, cookies ose për të modifikuar përmbajtjen e faqes.

Shembull: nëse një koment shfaqet pa escape-im si

```
<div>{koment}</div>
```

dhe komentit është `<script>fetch('/steal?c='+document.cookie)</script>`, skripti ekzekutohet.

Mbrojtja:

1. Escape output (React e bën automatikisht; në template-e të tjera përdor `htmlspecialchars` ose ekuivalent).
2. Content Security Policy (CSP) header që kufizon burimet e skripteve.
3. Validim dhe sanitizim input-i (p.sh. me `DOMPurify`).
4. Ruajtja e tokenit në `httpOnly` cookie që nuk lexohet nga JS.

Pyetja 10: Çfarë janë HTTPS dhe certifikatat SSL/TLS dhe pse janë të rëndësishme për transmetimin e tokenave?

Përgjigja:

HTTPS është versioni i sigurt i HTTP, ku komunikimi enkriptohet përmes protokollit TLS (pasardhësi i SSL).

Certifikata SSL/TLS lëshohet nga një Certificate Authority (CA) dhe vërteton identitetin e serverit, duke bërë të mundur enkriptimin e të dhënave që udhëtojnë midis klientit dhe serverit.

Pse është thelbësor për tokenat:

1. Pa HTTPS, JWT-të, cookies dhe passwords mund të përgjohen me sulme Man-in-the-Middle (p.sh. në WiFi publik).
2. Cookie me flag 'Secure' dërgohet vetëm përmes HTTPS.
3. Browser-at modernë i shënojnë faqet HTTP si 'Not Secure' dhe bllokojnë veçori të caktuara.

Certifikatat falas mund të merren përmes Let's Encrypt, dhe konfigurimi bëhet zakonisht në nivel reverse proxy (Nginx, Apache, Caddy).

4. Stilizimi (Bootstrap, Tailwind, CSS)

Pyetja 1: Çfarë është Bootstrap dhe çfarë ofron?

Përgjigja:

Bootstrap është një framework CSS me burim të hapur, i zhvilluar fillimisht nga Twitter, që përdoret për të ndërtuar shpejt ndërfaqe të stilizuara dhe responsive. Ofron një grid system me 12 kolona të bazuar në flexbox, një bibliotekë të gjerë komponentesh të gatshëm (Navbar, Modal, Card, Button, Form) dhe një grup utility classes për margin, padding, ngjyra dhe tekst.

Përfshihet zakonisht nëpërmjet CDN-së ose si paketë npm:

```
<link href='https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css'
rel='stylesheet'>
```

Pyetja 2: Si funksionon Grid System-i i Bootstrap-it?

Përgjigja:

Grid System-i i Bootstrap-it bazohet në tre elemente kryesore: container, row dhe col. Container-i është wrapper-i kryesor, row-i grupon kolonat dhe col-* përcakton gjerësinë e secilës kolonë në një rresht me 12 njësi. Klasat col-sm-*, col-md-*, col-lg-* dhe col-xl-* lejojnë sjellje të ndryshme në breakpoint-e të ndryshme.

Shembull:

```
<div class='container'>
  <div class='row'>
    <div class='col-md-6'>Gjysma majtas</div>
    <div class='col-md-6'>Gjysma djathtas</div>
  </div>
</div>
```

Pyetja 3: Cilat janë komponentet kryesore të Bootstrap-it?

Përgjigja:

Bootstrap ofron një gamë të gjerë komponentesh të gatshëm që përshpejtojnë zhvillimin e ndërfaqes:

1. Navbar: shirit navigimi në krye të faqes, me mbështetje për menu responsive.
2. Modal: dritare dialogu që shfaqet mbi përmbajtjen kryesore.
3. Card: kontejner fleksibël për të shfaqur informacion të grupuar (titull, tekst, imazh, butona).
4. Button: butona me variante ngjyrash (btn-primary, btn-success, btn-danger).
5. Form: input-e, label-e, select dhe checkbox të stilizuar me klasat form-control dhe form-check.

Shembull buton: <button class='btn btn-primary'>Ruaj</button>

Pyetja 4: Çfarë është Tailwind CSS dhe çfarë e dallon nga Bootstrap?

Përgjigja:

Tailwind CSS është një framework utility-first që nuk ofron komponente të gatshëm, por një grup të madh klasash utility që zbatojnë drejtpërdrejt veti CSS (p.sh. p-4 për padding, text-center për tekst të qendëruar). Zhvilluesi i kombinon këto klasa në HTML për të ndërtuar dizajn të personalizuar.

Dallimet kryesore nga Bootstrap:

1. Bootstrap ofron komponente të gatshme; Tailwind ofron blloqe ndërtimi.
2. Tailwind prodhon dizajn më unik, ndërsa Bootstrap ka pamje më të standardizuar.
3. Tailwind ka mjetin PurgeCSS që largon klasat e papërdorura, duke krijuar bundle shumë të vogël CSS.
4. Tailwind konfigurohet me skedarin tailwind.config.js për tema, ngjyra dhe breakpoint-e.

Pyetja 5: Cilët janë disa shembuj utility classes në Tailwind?

Përgjigja:

Tailwind ofron klasa utility për çdo veti CSS të zakonshme. Disa kategori dhe shembuj:

1. Spacing: p-4 (padding 1rem), m-2 (margin 0.5rem), px-6 (padding horizontal).
2. Layout: flex, grid, block, hidden.
3. Ngjyra: bg-blue-500 (background blu), text-white (tekst i bardhë), border-red-300.
4. Tipografi: text-xl, font-bold, text-center.
5. Responsive: md:w-1/2 (gjysmë gjerësie nga breakpoint-i md e lart), lg:flex-row.

Shembull kombinimi:

```
<div class='flex p-4 bg-blue-500 text-white md:w-1/2 rounded-lg'>  
  Karton i stilizuar  
</div>
```

Pyetja 6: Çfarë është responsive design dhe si arrihet?

Përgjigja:

Responsive design është qasja e ndërtimit të faqeve web që përshtaten automatikisht me madhësi të ndryshme ekrani: telefon, tablet, desktop. Arrihet kryesisht nëpërmjet media queries, breakpoint-eve dhe njësive relative (% , rem, vw).

Shembull media query në CSS:

```
@media (max-width: 768px) {  
  .menu { flex-direction: column; }  
}
```

Breakpoint-et tipike janë: 576px (sm), 768px (md), 992px (lg), 1200px (xl). Përveç kësaj, përdoret meta tag-u viewport në HTML:

```
<meta name='viewport' content='width=device-width, initial-scale=1'>
```

Pyetja 7: Çfarë është Flexbox dhe kur përdoret?

Përgjigja:

Flexbox (Flexible Box Layout) është një modul CSS që menaxhon shpërndarjen e elementeve përgjatë një boshti të vetëm, horizontal ose vertikal. Aktivizohet duke caktuar display: flex te elementi prind. Përdoret kur duhet të rreshtohen ose qendrohen elementet, të shpërndahet hapësira midis tyre ose të ndryshohet renditja vizuale.

Veti kryesore:

1. flex-direction: row ose column.
2. justify-content: shpërndarja përgjatë boshtit kryesor (center, space-between).
3. align-items: rreshtimi përgjatë boshtit tërthor (center, stretch).

Shembull:

```
.container { display: flex; justify-content: space-between; align-items: center; }
```

Pyetja 8: Cili është ndryshimi midis CSS Grid dhe Flexbox?

Përgjigja:

CSS Grid është system layout dy-dimensional që menaxhon njëkohësisht rreshta dhe kolona, ndërsa Flexbox është një-dimensional dhe punon në një bosht të vetëm (rresht ose kolonë).

Kur përdoret Grid: për layout të gjithë faqes, galeri imazhesh, dashboard-e me zona të ndryshme.

Kur përdoret Flexbox: për komponente të vegjël si navbar, listë butonash, qendrim vertikal/horizontal.

Shembull Grid:

```
.layout { display: grid; grid-template-columns: 200px 1fr; grid-template-rows: auto 1fr auto; }
```

Shembull Flexbox:

```
.navbar { display: flex; justify-content: space-between; align-items: center; }
```

Në praktikë, të dyja kombinohen: Grid për strukturën e jashtme dhe Flexbox brenda komponenteve.